

fastEmbedR Reference Manual

July 27, 2026

Type Package

Title Fast Native Embeddings and Graph Clustering from KNN Matrices

Version 0.99.0

Author Moussa Kassim [aut],
Martin Ocharo [aut],
Alessia Vignoli [aut],
Leonardo Tenori [aut],
Stefano Cacciatore [aut, cre]

Maintainer Stefano Cacciatore <tkcaccia@gmail.com>

Copyright See inst/COPYRIGHTS for the FAISS and Faiss-mlx derived native KNN components.

Description A compact UMAP and openTSNE-style implementation focused on fast embeddings from data matrices or precomputed nearest-neighbor index and distance matrices, with native KNN graph construction and Louvain, Leiden, and Walktrap community detection. Native float32 CPU HNSW and Apple Metal exact/IVF-Flat search support one-call workflows. Optional package-native CUDA search links directly to FAISS GPU for exact search and to the RAPIDS cuVS C API for IVF-Flat, keeping results on the device for CUDA embedding. Preprocessing, graph or affinity construction, fixed-reference transform, and stochastic layout optimization use native code with explicit CPU, Metal, and CUDA backend reporting.

License MIT + file LICENSE

URL <https://github.com/tkcaccia/fastEmbedR>

BugReports <https://github.com/tkcaccia/fastEmbedR/issues>

biocViews Software,
Preprocessing,
DimensionReduction,
Visualization,
Clustering,
SingleCell,
FlowCytometry,
GeneExpression,
GPU

Encoding UTF-8

Imports methods,
Rcpp,
withr

Suggests BiocStyle,

float,
igraph,
jsonlite,
knitr,
rmarkdown,
Rtsne,
testthat,
umap,
uwot

Enhances fastPLS ($\geq 0.99.3$)**LinkingTo** Rcpp

SystemRequirements C++17; optional Apple Metal framework on macOS for native Metal embedding backends; optional CUDA Toolkit with FAISS GPU, cuFFT, cuBLAS, cuSOLVER, RAPIDS RAFT, and RAPIDS cuVS C libraries for native CUDA embedding, PCA initialization, and nearest-neighbor backends

VignetteBuilder knitr**Config/roxygen2/version** 8.0.0**Config/testthat/edition** 3**Contents**

embed_knn	2
evaluate_embedding	3
fit_landmark_model	5
graph_cluster	6
knn_graph	7
landmark_tsne	8
landmark_umap	10
opentsne	11
opentsne_knn	13
opentsne_pca_init	15
pca	16
precompute_knn	17
precompute_query_knn	18
prepare_opentsne_knn	19
prepare_umap_knn	20
project_landmark_model	21
select_landmarks	22
transform_tsne	23
umap	25
umap_init	26
umap_knn	27

embed_knn

*Embed from precomputed KNN***Description**

Embed from precomputed KNN

Usage

```
embed_knn(
  indices,
  distances = NULL,
  method = "umap",
  n_components = 2L,
  seed = 4L,
  verbose = FALSE,
  backend = c("cpu", "cuda", "metal"),
  n_threads = NULL,
  ...
)
```

Arguments

indices	A list containing KNN ‘indices’ and ‘distances’, or an integer KNN index matrix. If a self-neighbor first column is present it is removed automatically.
distances	Numeric KNN distance matrix matching ‘indices’. Leave as ‘NULL’ when ‘indices’ is a KNN list.
method	Embedding method: “opentsne” or “umap”.
n_components	Output dimensionality.
seed	Random seed.
verbose	Print native optimizer progress.
backend	Execution backend: “cpu”, “cuda”, or “metal”.
n_threads	Number of CPU worker threads used by the CPU optimizer. Native GPU optimizers ignore this argument.
...	Additional openTSNE-specific parameters such as ‘perplexity’, iteration controls, ‘Y_init’, learning-rate controls, and exaggeration controls.

Value

A numeric embedding matrix with resolved settings stored in ‘attr(layout, "fastEmbedR_config”)’.

Examples

```
x <- scale(as.matrix(iris[1:30, 1:4]))
d <- as.matrix(stats::dist(x))
diag(d) <- Inf
k <- 5L
indices <- t(apply(d, 1L, order))[, seq_len(k), drop = FALSE]
distances <- matrix(
```

```

d[cbind(rep(seq_len(nrow(d)), each = k), as.vector(t(indices)))],
  nrow = nrow(d), byrow = TRUE
)
layout <- embed_knn(indices, distances, method = "umap", seed = 1)

```

evaluate_embedding	<i>Evaluate an embedding against high-dimensional structure</i>
--------------------	---

Description

Trustworthiness and continuity use their standard rank-penalty definitions. Local rank metrics are computed exactly on a deterministic subsample so the function does not allocate all-pairs matrices for the complete dataset. Float32 inputs are sampled before conversion to a dense double matrix.

Usage

```

evaluate_embedding(
  x_high,
  embedding,
  labels = NULL,
  batch = NULL,
  k = c(15L, 30L, 50L),
  primary_k = NULL,
  reference_nn = NULL,
  sample_size_for_global_metrics = min(5000L, nrow(x_high)),
  sample_size_for_local_metrics = min(2000L, nrow(x_high)),
  use_cache = FALSE,
  cache_dir = file.path("results", "cache"),
  force_recompute = FALSE,
  seed = NA_integer_,
  method = NA_character_,
  backend = NA_character_,
  n_threads = NULL,
  dataset = NA_character_
)

```

Arguments

<code>x_high</code>	High-dimensional input, usually scaled data or PCA scores.
<code>embedding</code>	Low-dimensional embedding matrix.
<code>labels</code>	Optional biological/class labels.
<code>batch</code>	Optional batch labels.
<code>k</code>	Neighbor sizes used for structure metrics.
<code>primary_k</code>	Neighbor size used for scalar trustworthiness, continuity, label accuracy, rare-class recall, density summaries, and rank-error metrics. The default 'NULL' preserves the historical <code>k = 15</code> behavior when possible.
<code>reference_nn</code>	Optional precomputed high-dimensional nearest-neighbor list. It may include a self-neighbor column.
<code>sample_size_for_global_metrics</code>	Maximum deterministic subsample size for all-pairs global distance metrics.

sample_size_for_local_metrics	Maximum deterministic subsample size for local trustworthiness, continuity, and neighbor preservation metrics.
use_cache	Cache the high-dimensional reference neighbors used by quality metrics.
cache_dir	Directory for quality-metric reference-neighbor cache files.
force_recompute	Ignore cached quality reference neighbors.
seed	Random seed recorded in the output and used for subsampling.
method	Method name recorded in the output.
backend	Backend name recorded in the output.
n_threads	Number of CPU worker threads used when quality metrics need nearest-neighbor searches. Ignored by native GPU metric backends.
dataset	Dataset name recorded in the output.

Value

A one-row data frame with local, global, label-aware, batch-aware, and metadata columns. Per-class recall is also attached as an attribute.

Examples

```
x <- scale(as.matrix(iris[, 1:4]))
metrics <- evaluate_embedding(
  x, x[, 1:2], labels = iris$Species, k = 5,
  sample_size_for_local_metrics = 100,
  sample_size_for_global_metrics = 100,
  seed = 1
)
```

fit_landmark_model	<i>Fit an embedding model on selected landmarks</i>
--------------------	---

Description

`fit_landmark_model()` runs the ordinary production [umap()] or [opentsne()] implementation on the selected reference rows. The returned object stores the reference data and embedding needed to project the complementary rows or genuinely new observations.

Usage

```
fit_landmark_model(
  data,
  selection,
  method = c("umap", "opentsne"),
  n_neighbors = NULL,
  perplexity = NULL,
  n_components = 2L,
  metric = c("euclidean", "cosine", "correlation", "inner_product"),
  seed = 4L,
  backend = c("cpu", "cuda", "metal"),
```

```

    n_threads = NULL,
    graph_mode = c("binary", "fuzzy"),
    keep_knn = FALSE,
    verbose = FALSE,
    ...
  )

```

Arguments

data	Data in the analysis space used for KNN. Preprocess once before calling this staged API when centering, scaling, or PCA is required.
selection	A result from [select_landmarks()] or explicit landmark row indices.
method	"umap" or "opentsne".
n_neighbors	UMAP neighborhood size. For openTSNE this is the precomputed KNN support width; 'NULL' derives it from 'perplexity'.
perplexity	openTSNE perplexity.
n_components	Embedding dimensionality.
metric	KNN metric.
seed	Random seed.
backend	"cpu", "cuda", or "metal".
n_threads	CPU worker threads.
graph_mode	UMAP graph mode, passed unchanged to [umap()].
keep_knn	Retain reference KNN output.
verbose	Print optimizer progress.
...	Additional optimizer arguments passed to [opentsne()].

Value

A 'fastEmbedR_landmark_model'.

Examples

```

x <- scale(as.matrix(iris[, 1:4]))
selection <- select_landmarks(x, 0.5, seed = 1)
model <- fit_landmark_model(
  x, selection, method = "umap", n_neighbors = 10,
  graph_mode = "fuzzy", seed = 1
)

```

graph_cluster

Native graph community detection

Description

Runs native C++ graph clustering without calling Python, igraph, cuGraph, or an external clustering routine. Walktrap is the Pons-Latapy random-walk method and is memory-guarded because its exact transition matrix is quadratic.

Usage

```
graph_cluster(
  graph,
  method = c("leiden", "louvain", "walktrap"),
  resolution = 1,
  n_iterations = 10L,
  n_runs = 1L,
  steps = 4L,
  seed = 1L
)
```

Arguments

graph	A fastEmbedR_graph or compatible weighted edge list.
method	Native Leiden, Louvain, or exact Walktrap.
resolution	Positive modularity resolution for Leiden and Louvain.
n_iterations	Maximum local-moving and Leiden iterations.
n_runs	Independent Louvain or Leiden runs.
steps	Random-walk length for Walktrap.
seed	Random seed for Louvain and Leiden.

Value

A fastEmbedR_graph_cluster list with one-based membership, modularity, implementation meta-data, and timing.

References

Blondel VD, Guillaume JL, Lambiotte R, Lefebvre E. Fast unfolding of communities in large networks. J Stat Mech. 2008;2008:P10008.

Traag VA, Waltman L, van Eck NJ. From Louvain to Leiden: guaranteeing well-connected communities. Sci Rep. 2019;9:5233.

Pons P, Latapy M. Computing communities in large networks using random walks. J Graph Algorithms Appl. 2006;10:191-218.

Examples

```
x <- scale(as.matrix(iris[, 1:4]))
graph <- knn_graph(x, k = 15)
clusters <- graph_cluster(graph, method = "leiden")
table(clusters$membership)
```

knn_graph

*Build a compact nearest-neighbor graph***Description**

Builds one undirected weighted graph from data, an embedding, or precomputed neighbors. The function reuses fastEmbedR's existing KNN route and does not expose another nearest-neighbor tuning surface.

Usage

```
knn_graph(
  x,
  k = 30L,
  backend = c("cpu", "cuda", "metal"),
  metric = c("euclidean", "cosine", "correlation"),
  weight = c("snn", "distance", "binary"),
  mutual = FALSE,
  prune = 0,
  n_threads = 1L
)
```

Arguments

x	Numeric data, a fastEmbedR_embedding, a precomputed KNN list, or an existing fastEmbedR_graph.
k	Number of non-self neighbors retained.
backend	KNN-search backend used only for data input.
metric	Euclidean, cosine, or correlation distance.
weight	SNN/Jaccard, inverse-distance, or binary edge weights.
mutual	Whether to retain only reciprocal KNN edges.
prune	Non-negative edge-weight pruning threshold.
n_threads	CPU threads for search and graph construction.

Value

A compact fastEmbedR_graph edge-list object.

Examples

```
x <- scale(as.matrix(iris[, 1:4]))
graph <- knn_graph(x, k = 15)
graph
```

landmark_tsne	<i>Landmark t-SNE with openTSNE-style transform</i>
---------------	---

Description

`'landmark_tsne()'` embeds a subset of observations with `[opentsne()]`, then places the remaining observations with `'transform_tsne()'`. This mirrors the practical landmark workflow exposed by openTSNE while keeping the implementation native to this package. Projection KNN searches only the fixed landmark reference: CPU uses native HNSW, Metal uses native exact or recall-tuned IVF-Flat, and CUDA keeps native exact or IVF-Flat results resident on the device.

Usage

```
landmark_tsne(
  data,
  landmarks = TRUE,
  reference_method = c("opentsne"),
  n_neighbors = NULL,
  perplexity = NULL,
  n_components = 2L,
  standardize = TRUE,
  pca_dims = NULL,
  seed = 4L,
  backend = c("cpu", "cuda", "metal"),
  transform_k = NULL,
  transform_perplexity = 5,
  transform_iter = 250L,
  transform_early_exaggeration_iter = 0L,
  transform_n_negatives = NULL,
  initialization = c("median", "weighted", "random"),
  keep_knn = FALSE,
  verbose = FALSE,
  n_threads = NULL,
  ...
)
```

Arguments

<code>data</code>	Numeric matrix/data frame with observations in rows.
<code>landmarks</code>	<code>'TRUE'</code> for an automatic subset, a fraction such as <code>'0.5'</code> , a landmark count, or explicit row indices.
<code>reference_method</code>	Kept for compatibility. Only <code>"opentsne"</code> is accepted in the cleaned package API.
<code>n_neighbors</code>	Number of non-self neighbors used to embed the landmark reference set. If <code>'NULL'</code> , it follows the same neighbor policy as <code>[opentsne()]</code> : <code>'ceiling(perplexity)'</code> when perplexity is supplied.
<code>perplexity</code>	t-SNE perplexity for the landmark reference embedding. If <code>'NULL'</code> , the optimizer chooses a safe value from the reference KNN width and sample size.
<code>n_components</code>	Output dimensionality, from 1 to 3.

standardize	Center and scale columns before KNN. Defaults to 'FALSE' so one-call results match a KNN object computed from the supplied matrix.
pca_dims	Optional PCA dimension before KNN.
seed	Random seed.
backend	Execution backend: "cpu", "cuda", or "metal".
transform_k	Number of landmark neighbors used to place non-landmarks.
transform_perplexity	Perplexity used by 'transform_tsne()'.
transform_iter	Number of normal transform iterations. Use '0' for projection-only landmarking with no transform refinement.
transform_early_exaggeration_iter	Number of transform early exaggeration iterations.
transform_n_negatives	Number of sampled reference negatives used by 'transform_tsne()' on large landmark sets. GPU sampled transform repulsion is native and experimental for large reference sets.
initialization	Initial placement for transformed observations.
keep_knn	If 'TRUE', retain KNN matrices in the returned object.
verbose	Print optimizer progress.
n_threads	Number of CPU worker threads used by CPU KNN and CPU transform optimization. Native GPU stages ignore this argument.
...	Additional low-level parameters passed to [opentsne_knn()].

Value

A 'fastEmbedR_embedding' object.

Examples

```
fit <- landmark_tsne(
  as.matrix(iris[, 1:4]), landmarks = 0.5, perplexity = 5,
  early_exaggeration_iter = 5, n_iter = 10,
  transform_iter = 5, seed = 1
)
plot(fit, labels = iris$Species)
```

landmark_umap

Run Landmark UMAP

Description

Embed a landmark subset with [umap](#) and project the remaining observations against the fixed landmark embedding using query-to-reference KNN, local affine placement, and optional fixed-reference refinement.

Usage

```
landmark_umap(
  data,
  landmarks = 0.5,
  n_neighbors = NULL,
  n_components = 2L,
  standardize = TRUE,
  pca_dims = NULL,
  seed = 4L,
  backend = c("cpu", "cuda", "metal"),
  transform_k = NULL,
  n_threads = NULL,
  keep_knn = FALSE,
  graph_mode = c("binary", "fuzzy"),
  verbose = FALSE
)
```

Arguments

data	Numeric matrix or data frame with observations in rows.
landmarks	TRUE for an automatic subset, a fraction such as 0.5, a landmark count, or explicit row indices.
n_neighbors	Number of non-self neighbors.
n_components	Output dimensionality.
standardize	Center and scale columns before KNN.
pca_dims	Optional PCA dimension before KNN.
seed	Random seed.
backend	Execution backend: "cpu", "cuda", or "metal". CPU uses native HNSW; Metal uses native exact/IVF-Flat; CUDA requests package-native RAPIDS cuVS KNN with target_recall = 0.99. GPU requests must use a real native backend and are not silently relabelled as CPU.
transform_k	Number of landmark neighbors used to project non-landmark observations. Defaults to n_neighbors.
n_threads	Number of CPU worker threads.
keep_knn	Keep projection KNN matrices in the returned object.
graph_mode	Graph weighting mode passed unchanged to the reference umap fit. "binary" uses a symmetric unit-weight graph and "fuzzy" uses standard UMAP fuzzy graph weights.
verbose	Print progress.

Value

A fastEmbedR_embedding object with the full layout, landmark metadata, timings, and parameters.

Examples

```
x <- scale(as.matrix(iris[, 1:4]))
fit <- landmark_umap(x, landmarks = 0.5, n_neighbors = 15, seed = 1)
plot(fit)
```

opentsne

*Run native openTSNE-style t-SNE from a data matrix***Description**

‘opentsne()’ computes or reuses a KNN graph, then runs the package-native openTSNE-style two-phase optimizer. The default optimizer is CPU-native. Explicit ‘backend = "metal"’ and ‘backend = "cuda"’ requests use the matching package-native GPU optimizer when compiled and fail clearly if the requested backend is unavailable. "openTSNE-style" describes the published sparse-affinity, interpolation-based t-SNE workflow, not Python API, object, default, or coordinate compatibility. No Python openTSNE code is called.

Usage

```
opentsne(
  data,
  perplexity = NULL,
  n_components = 2L,
  init_data = NULL,
  Y_init = NULL,
  standardize = FALSE,
  pca_dims = NULL,
  metric = c("euclidean", "cosine", "correlation", "inner_product"),
  nn = NULL,
  seed = 4L,
  backend = c("cpu", "cuda", "metal"),
  keep_knn = FALSE,
  verbose = FALSE,
  n_threads = NULL,
  learning_rate = "auto",
  early_exaggeration_iter = NULL,
  early_exaggeration = "auto",
  n_iter = NULL,
  exaggeration = NULL,
  initial_momentum = 0.8,
  final_momentum = 0.8,
  max_step_norm = "auto",
  negative_gradient_method = "auto",
  record_costs = FALSE,
  auto_config = TRUE,
  ...
)
```

Arguments

data	Numeric matrix/data frame with observations in rows, or a list containing KNN ‘indices’ and ‘distances’.
perplexity	t-SNE perplexity. The one-call API uses ‘ceiling(perplexity)’ non-self neighbors internally. If ‘NULL’, uses the largest safe value up to 30 that is available for the input.

n_components	Output dimensionality, from 1 to 3.
init_data	Optional original high-dimensional data matrix used only to compute PCA initialization with [opentsne_pca_init()]. It is not used for neighbor search or optimization.
Y_init	Optional explicit initial layout. Use [opentsne_pca_init()] to precompute and reuse a PCA initialization.
standardize	Center and scale columns before KNN. Defaults to 'FALSE' so one-call results match a KNN object computed from the supplied matrix.
pca_dims	Optional PCA dimension before KNN.
metric	KNN distance metric for one-call matrix input: "euclidean", "cosine", "correlation", or "inner_product".
nn	Optional precomputed KNN output when 'data' is a data matrix.
seed	Random seed.
backend	Execution backend: "cpu", "cuda", or "metal". CPU KNN uses package-native HNSW. Metal uses package-native exact or recall-tuned IVF-Flat search. CUDA uses direct FAISS GPU exact search below 100,000 rows and direct RAPIDS cuVS IVF-Flat above that threshold, then passes the device pointers directly to native CUDA openTSNE. Unsupported GPU requests fail clearly and are not relabelled CPU runs.
keep_knn	If 'TRUE', retain KNN matrices in the returned object.
verbose	Print optimizer progress.
n_threads	Number of CPU worker threads used by CPU KNN and CPU openTSNE optimization. Native GPU optimizers ignore this argument.
learning_rate	Positive number or "auto". With "auto", the native optimizer uses 'n / exaggeration' separately for each phase.
early_exaggeration_iter	Number of early-exaggeration iterations.
early_exaggeration	Early-exaggeration multiplier, or "auto" for openTSNE's default rule.
n_iter	Number of normal optimization iterations after early exaggeration.
exaggeration	Normal-phase exaggeration. 'NULL' means 1.
initial_momentum	Momentum during early exaggeration.
final_momentum	Momentum during normal optimization.
max_step_norm	Maximum per-point update norm. "auto" uses the standard CPU limit and a tighter native Metal FFT-grid limit to avoid float32 outlier steps. Use 'NULL' or 'NA' to disable clipping.
negative_gradient_method	"auto", "exact", or "fft". On CPU, "auto" resolves to the native grid-FFT Flt-SNE-style negative-gradient approximation. Native GPU FFT/exact paths are used only when the corresponding compiled symbols are available; otherwise GPU requests fail clearly rather than falling back to CPU.
record_costs	If 'TRUE', compute diagnostic KL/cost traces.
auto_config	If 'TRUE', choose missing t-SNE settings with a native C++ opt-SNE-inspired policy. The policy uses 'n / early_exaggeration' for "auto" learning rate, chooses missing iteration limits, and enables KLD-based early stopping only on CPU/small exact runs where the monitor is not prohibitively expensive. Explicit user-supplied values are respected.
...	Additional low-level parameters passed to [opentsne_knn()].

Value

A ‘fastEmbedR_embedding’ object.

Examples

```
fit <- opentsne(
  as.matrix(iris[, 1:4]), perplexity = 5,
  early_exaggeration_iter = 5, n_iter = 10, seed = 1
)
plot(fit, labels = iris$Species)
```

opentsne_knn

Run native openTSNE-style t-SNE from precomputed KNN

Description

‘opentsne_knn()’ is the direct KNN-input entry point for the native openTSNE-style optimizer. It accepts either a list containing KNN ‘indices’ and ‘distances’ or separate KNN index and distance matrices. No neighbor search, scaling, or PCA is done inside this function. Here, “openTSNE-style” describes algorithmic lineage, not compatibility with the Python package. fastEmbedR defines its own R API, defaults, objects, and native optimizer kernels; it does not call or port Python openTSNE.

Usage

```
opentsne_knn(
  indices,
  distances = NULL,
  n_neighbors = NULL,
  perplexity = NULL,
  n_components = 2L,
  init_data = NULL,
  Y_init = NULL,
  seed = 4L,
  verbose = FALSE,
  backend = c("cpu", "cuda", "metal"),
  n_threads = NULL,
  learning_rate = "auto",
  early_exaggeration_iter = NULL,
  early_exaggeration = "auto",
  n_iter = NULL,
  exaggeration = NULL,
  initial_momentum = 0.8,
  final_momentum = 0.8,
  max_step_norm = "auto",
  negative_gradient_method = "auto",
  record_costs = FALSE,
  auto_config = TRUE,
  ...
)
```

Arguments

<code>indices</code>	A list containing KNN ‘indices’ and ‘distances’, or an integer KNN index matrix.
<code>distances</code>	Numeric KNN distance matrix matching ‘indices’. Leave ‘NULL’ when ‘indices’ is a KNN list.
<code>n_neighbors</code>	Optional number of non-self neighbor columns to use from the supplied KNN graph. This lets you compute a wide KNN once and reuse its first columns for comparable tests.
<code>perplexity</code>	t-SNE perplexity. If ‘NULL’, the optimizer chooses a safe value from the supplied KNN width and sample size.
<code>n_components</code>	Output dimensionality, from 1 to 3.
<code>init_data</code>	Optional original high-dimensional data matrix used only to compute PCA initialization for KNN-input runs. It is not used for neighbor search or optimization.
<code>Y_init</code>	Optional explicit initial layout. Use <code>[opentsne_pca_init()]</code> to precompute and reuse a PCA initialization.
<code>seed</code>	Random seed.
<code>verbose</code>	Print optimizer progress.
<code>backend</code>	Execution backend: “cpu”, “cuda”, or “metal”. CPU KNN uses package-native HNSW. Metal uses package-native exact or recall-tuned IVF-Flat search. CUDA uses direct FAISS GPU exact search below 100,000 rows and direct RAPIDS cuVS IVF-Flat above that threshold, then passes the device pointers directly to native CUDA openTSNE. Unsupported GPU requests fail clearly and are not relabelled CPU runs.
<code>n_threads</code>	Number of CPU worker threads used by CPU KNN and CPU openTSNE optimization. Native GPU optimizers ignore this argument.
<code>learning_rate</code>	Positive number or “auto”. With “auto”, the native optimizer uses ‘n / exaggeration’ separately for each phase.
<code>early_exaggeration_iter</code>	Number of early-exaggeration iterations.
<code>early_exaggeration</code>	Early-exaggeration multiplier, or “auto” for openTSNE’s default rule.
<code>n_iter</code>	Number of normal optimization iterations after early exaggeration.
<code>exaggeration</code>	Normal-phase exaggeration. ‘NULL’ means 1.
<code>initial_momentum</code>	Momentum during early exaggeration.
<code>final_momentum</code>	Momentum during normal optimization.
<code>max_step_norm</code>	Maximum per-point update norm. “auto” uses the standard CPU limit and a tighter native Metal FFT-grid limit to avoid float32 outlier steps. Use ‘NULL’ or ‘NA’ to disable clipping.
<code>negative_gradient_method</code>	“auto”, “exact”, or “fft”. On CPU, “auto” resolves to the native grid-FFT Flt-SNE-style negative-gradient approximation. Native GPU FFT/exact paths are used only when the corresponding compiled symbols are available; otherwise GPU requests fail clearly rather than falling back to CPU.
<code>record_costs</code>	If ‘TRUE’, compute diagnostic KL/cost traces.

`auto_config` If 'TRUE', choose missing t-SNE settings with a native C++ opt-SNE-inspired policy. The policy uses 'n/early_exaggeration' for '"auto"' learning rate, chooses missing iteration limits, and enables KLD-based early stopping only on CPU/small exact runs where the monitor is not prohibitively expensive. Explicit user-supplied values are respected.

... Additional low-level parameters passed to `[opentsne_knn()]`.

Value

A numeric embedding matrix with settings stored in 'attr(layout, "fastEmbedR_config")'.

Examples

```
x <- scale(as.matrix(iris[, 1:4]))
d <- as.matrix(stats::dist(x))
diag(d) <- Inf
k <- 15L
idx <- t(apply(d, 1L, order))[, seq_len(k), drop = FALSE]
dst <- matrix(d[cbind(rep(seq_len(nrow(d)), each = k), as.vector(t(idx)))],
  nrow = nrow(d), byrow = TRUE)
layout <- opentsne_knn(idx, dst, init_data = x, perplexity = 5,
  early_exaggeration_iter = 5, n_iter = 10)
```

opentsne_pca_init	<i>Compute or reuse a PCA initialization for openTSNE</i>
-------------------	---

Description

'opentsne_pca_init()' creates the small-scale PCA initialization used by `[opentsne()]` and `[opentsne_knn()]`. Supplying 'cache_file' stores the result as an RDS file; later calls with the same path reuse the saved matrix instead of recomputing PCA. This is useful when comparing several KNN backends with exactly the same initialization. For 'backend = "cuda"', fastEmbedR requires native RAPIDS RAFT/cuML TSVD support compiled into the CUDA backend and fails loudly if that backend is unavailable. Metal uses fastEmbedR's native float32 block-subspace TSVD with native Metal centering/scaling, Metal Performance Shaders matrix products, and one resident GPU workspace. CPU prefers 'fastPLS >= 0.99.3' RSVD when it is installed and otherwise uses the package-local RSVD helper. The package does not call Python or 'reticulate' for PCA initialization.

Usage

```
opentsne_pca_init(
  data,
  n_components = 2L,
  seed = 4L,
  backend = c("cpu", "metal", "cuda"),
  cache_file = NULL,
  force_recompute = FALSE
)
```

Arguments

data	Numeric matrix/data frame or 'float::float32' matrix with observations in rows.
n_components	Output dimensionality, usually '2'.
seed	Random seed used by the truncated PCA subspace sketch.
backend	Backend used for PCA when available: "cpu", "metal", or "cuda".
cache_file	Optional '.rds' file path. If it exists and 'force_recompute = FALSE', the saved initialization is loaded and validated.
force_recompute	If 'TRUE', ignore any existing cache and recompute.

Value

An initialization matrix suitable for 'Y_init'. Float32 input is returned as 'float::float32' when the selected PCA backend preserves it.

Examples

```
init <- opentsne_pca_init(as.matrix(iris[, 1:4]), seed = 1)
plot(init, pch = 21, bg = iris$Species)
```

pca	<i>Backend-native truncated PCA</i>
-----	-------------------------------------

Description

pca() computes principal component scores with a backend-native truncated decomposition. CPU uses the fastPLS-style randomized SVD family. Metal uses a package-native float32 block-subspace TSVD whose large matrix products are executed with Metal Performance Shaders while data and work buffers remain resident in unified GPU memory. With float::float32 input, native Metal preprocessing and the returned scores/loadings also remain float32. The API intentionally has no decomposition method menu and does not call Python. Explicit unavailable GPU requests fail instead of being reported as GPU work.

Usage

```
pca(x, ncomp = 2L, xtest = NULL, center = TRUE, scale = FALSE,
    backend = c("cpu", "cuda", "metal"), seed = 4L,
    opentsne_init = FALSE)
```

Arguments

x	Numeric matrix/data frame or float::float32 matrix with observations in rows.
ncomp	Number of principal components.
xtest	Optional test/query matrix with the same columns as x. Projected coordinates are returned in scores_test.
center	If TRUE, mean-center columns before decomposition.
scale	If TRUE, scale centered columns to unit sample standard deviation before decomposition.

backend	PCA backend: "cpu", "cuda", or "metal".
seed	Random seed for the Gaussian subspace sketch.
opentsne_init	If TRUE, add opentsne_init to the returned PCA object. This matrix is centered and rescaled so its largest component standard deviation is 1e-4, ready to pass as Y_init to opentsne() or opentsne_knn().

Value

A fastEmbedR_pca list with scores, loadings, singular_values, centering/scaling vectors, backend metadata, and decomposition metadata. When opentsne_init = TRUE, the list also contains opentsne_init; when xtest is supplied, it also contains scores_test. Metal preserves float::float32 scores, loadings, initialization, and compatible test projections when the input is float32.

Examples

```
fit <- pca(
  as.matrix(iris[, 1:4]),
  ncomp = 2,
  seed = 1,
  opentsne_init = TRUE
)
plot(fit$scores, pch = 21, bg = iris$Species)
plot(fit$opentsne_init, pch = 21, bg = iris$Species)
```

precompute_knn	<i>Precompute native nearest neighbors</i>
----------------	--

Description

‘precompute_knn()’ exposes the same package-native nearest-neighbor search used internally by [umap()] and [opentsne()]. The search algorithm is chosen by fastEmbedR for the requested backend and is deliberately not a user parameter.

Usage

```
precompute_knn(
  data,
  k = 30L,
  metric = c("euclidean", "cosine", "correlation", "inner_product"),
  backend = c("cpu", "cuda", "metal"),
  n_threads = NULL
)
```

Arguments

data	Numeric matrix, numeric data frame, or a ‘float::float32’ matrix with observations in rows.
k	Number of non-self nearest neighbors to return.
metric	Distance metric: “euclidean”, “cosine”, “correlation”, or “inner_product”. Raw inner product is available only in CUDA builds that support it.
backend	Search backend: “cpu”, “cuda”, or “metal”.
n_threads	Number of CPU worker threads. Native GPU backends ignore this argument.

Details

CPU search uses the package-native recall-tuned HNSW implementation. Metal uses native exact search for small inputs and recall-tuned IVF-Flat for larger inputs. CUDA uses package-native direct FAISS GPU exact search below 100,000 observations and direct RAPIDS cuVS IVF-Flat above that threshold. The internal recall target is 0.99.

The CUDA result remains on the GPU and can be passed directly to [umap_knn()] or [opentsne_knn()] with 'backend = "cuda"'. CPU and Metal results contain one-based 'indices' and 'distances' matrices. Every result excludes the observation itself. An unavailable requested backend raises an error; no CPU fallback is reported as GPU work.

Value

A 'fastEmbedR_knn' object. Host results contain 'indices' and 'distances'; CUDA results additionally inherit from 'fastEmbedR_gpu_knn' and own device-resident index and distance buffers.

Examples

```
x <- scale(as.matrix(iris[, 1:4]))
knn <- precompute_knn(x, k = 15, backend = "cpu", n_threads = 2)
layout <- umap_knn(knn, backend = "cpu", seed = 1)
```

```
precompute_query_knn    Precompute query-to-reference nearest neighbors
```

Description

'precompute_query_knn()' searches a fixed reference matrix for every row of a query matrix. It uses the same package-native backend family and routing policy as [precompute_knn()], but does not compute unnecessary reference-to-reference or query-to-query neighbors.

Usage

```
precompute_query_knn(
  reference,
  query,
  k = 30L,
  metric = c("euclidean", "cosine", "correlation", "inner_product"),
  backend = c("cpu", "cuda", "metal"),
  n_threads = NULL
)
```

Arguments

reference	Reference observations in rows.
query	Query observations in rows and the same feature space as 'reference'.
k	Number of non-self nearest neighbors to return.
metric	Distance metric: "euclidean", "cosine", "correlation", or "inner_product". Raw inner product is available only in CUDA builds that support it.
backend	Search backend: "cpu", "cuda", or "metal".
n_threads	Number of CPU worker threads. Native GPU backends ignore this argument.

Details

CPU uses the native recall-tuned HNSW reference-query path. Metal routes between a native query-only exact kernel and recall-tuned IVF-Flat from the estimated reference-query distance workload. CUDA uses direct FAISS GPU exact search below 100,000 reference rows and direct RAPIDS cuVS IVF-Flat above that threshold. CUDA results remain device-resident for direct consumption by landmark UMAP and openTSNE transformations.

Value

A ‘fastEmbedR_knn’ object with one row per query observation and one-based indices into ‘reference’.

Examples

```
x <- scale(as.matrix(iris[, 1:4]))
ref <- x[1:100, , drop = FALSE]
qry <- x[101:150, , drop = FALSE]
knn <- precompute_query_knn(ref, qry, k = 10, backend = "cpu")
```

```
prepare_opentsne_knn    Precompute reusable openTSNE KNN state
```

Description

prepare_opentsne_knn() strips self-neighbors, trims to a compact non-self KNN width, and stores the materialized KNN matrices once. The current native optimizer still builds t-SNE affinities internally from this compact KNN state.

Usage

```
prepare_opentsne_knn(
  indices,
  distances = NULL,
  n_neighbors = NULL,
  perplexity = NULL
)
```

Arguments

indices	A KNN object, or an integer KNN index matrix.
distances	Numeric KNN distance matrix matching indices. Leave NULL when indices is a KNN object.
n_neighbors	Optional number of non-self neighbor columns to retain.
perplexity	Optional t-SNE perplexity used to choose the retained KNN width when n_neighbors is not supplied.

Value

A prepared openTSNE KNN object. Pass it to opentsne_knn() for repeated runs without repeating KNN normalization/materialization.

Examples

```
x <- scale(as.matrix(iris[, 1:4]))
d <- as.matrix(stats::dist(x))
diag(d) <- Inf
k <- 10L
idx <- t(apply(d, 1L, order))[, seq_len(k), drop = FALSE]
dst <- matrix(d[cbind(rep(seq_len(nrow(d)), each = k), as.vector(t(idx)))],
  nrow = nrow(d), byrow = TRUE)
prep <- prepare_opentsne_knn(idx, dst, perplexity = 5)
y <- opentsne_knn(prepare, early_exaggeration_iter = 10, n_iter = 20)
plot(y, pch = 21, bg = iris$Species)
```

prepare_umap_knn	<i>Precompute reusable UMAP graph state from KNN</i>
------------------	--

Description

prepare_umap_knn() builds the reusable UMAP graph once from a KNN object or index/distance matrices. This is useful for repeated seeds, benchmark grids, and CPU/GPU comparisons where the neighbor graph should remain fixed.

Usage

```
prepare_umap_knn(
  indices,
  distances = NULL,
  backend = c("cpu", "cuda", "metal"),
  n_threads = NULL,
  graph_mode = c("binary", "fuzzy")
)
```

Arguments

indices	A KNN object, or an integer KNN index matrix.
distances	Numeric KNN distance matrix matching indices. Leave NULL when indices is a KNN object.
backend	Execution backend used to resolve size-aware defaults: "cpu", "cuda", or "metal".
n_threads	Number of CPU worker threads used during graph construction.
graph_mode	Graph weighting mode. "binary" uses a symmetric unit-weight graph. "fuzzy" uses standard UMAP fuzzy graph weights.

Value

A prepared UMAP object containing normalized KNN data, a CSR graph, graph weights, and epochs_per_sample. Pass it to umap_knn() for repeated runs without rebuilding the graph.

Examples

```
x <- scale(as.matrix(iris[, 1:4]))
d <- as.matrix(stats::dist(x))
diag(d) <- Inf
k <- 10L
idx <- t(apply(d, 1L, order))[, seq_len(k), drop = FALSE]
dst <- matrix(d[cbind(rep(seq_len(nrow(d)), each = k), as.vector(t(idx)))],
  nrow = nrow(d), byrow = TRUE)
prep <- prepare_umap_knn(idx, dst)
y <- umap_knn(prepare, seed = 1)
plot(y, pch = 21, bg = iris$Species)
```

project_landmark_model

Project observations into a fitted landmark embedding

Description

‘project_landmark_model()’ computes query-to-reference KNN with the same native backend policy used by the full embedding functions, then applies the method-specific fixed-reference transform. Landmark coordinates remain fixed.

Usage

```
project_landmark_model(
  model,
  data,
  query_indices = NULL,
  transform_k = NULL,
  refinement_epochs = 50L,
  transform_perplexity = 5,
  transform_iter = 250L,
  transform_early_exaggeration_iter = 0L,
  transform_n_negatives = NULL,
  initialization = c("median", "weighted", "random"),
  keep_knn = FALSE,
  n_threads = NULL,
  verbose = FALSE,
  ...
)
```

Arguments

model	A model returned by [fit_landmark_model()].
data	The original full data matrix, or a matrix containing only new query observations in the same feature space.
query_indices	Optional rows of ‘data’ to project.
transform_k	Number of reference neighbors.
refinement_epochs	Fixed-reference UMAP refinement epochs.

```

transform_perplexity      Perplexity of the openTSNE transform.
transform_iter            Number of openTSNE transform iterations.
transform_early_exaggeration_iter  Transform exaggeration iterations.
transform_n_negatives      Number of reference negatives for openTSNE.
initialization            openTSNE query initialization.
keep_knn                  Retain query-to-reference KNN output.
n_threads                  CPU worker threads.
verbose                    Print optimizer progress.
...                        Low-level openTSNE transform controls.

```

Value

A ‘fastEmbedR_embedding’. When ‘data’ is the original full matrix, ‘layout’ is reassembled in original row order. Otherwise it contains only the supplied query rows.

Examples

```

x <- scale(as.matrix(iris[, 1:4]))
selection <- select_landmarks(x, 0.5, seed = 1)
model <- fit_landmark_model(
  x, selection, method = "umap", n_neighbors = 10, seed = 1
)
fit <- project_landmark_model(model, x, refinement_epochs = 2)

```

select_landmarks	<i>Select representative landmark observations</i>
------------------	--

Description

‘select_landmarks()’ separates row indices into a representative landmark reference and its complementary query set. It does not embed or project the data.

Usage

```
select_landmarks(data, landmarks = TRUE, seed = 4L, n_threads = NULL)
```

Arguments

```

data          Numeric matrix, data frame, or ‘float::float32’ matrix.
landmarks     ‘TRUE’ for an automatic count, a fraction in ‘(0, 1)’, a landmark count, or
              explicit row indices.
seed          Random seed used by projection-based landmark selection.
n_threads     Number of CPU threads used for float32 selection features.

```

Value

A ‘fastEmbedR_landmark_selection’ object containing ‘indices’ and ‘query_indices’.

Examples

```
x <- scale(as.matrix(iris[, 1:4]))
selection <- select_landmarks(x, landmarks = 0.5, seed = 1)
selection$indices
```

transform_tsne

Transform query points into an existing t-SNE embedding

Description

‘transform_tsne()’ places query observations into a fixed reference t-SNE embedding. It follows the openTSNE transform design at the algorithm level: query-to-reference affinities are computed from KNN distances, query points are initialized from nearby reference points, and optimization moves only the query points.

Usage

```
transform_tsne(
  reference_layout,
  knn = NULL,
  reference_data = NULL,
  new_data = NULL,
  k = NULL,
  perplexity = 5,
  initialization = c("median", "weighted", "random"),
  Y_init = NULL,
  n_iter = 250L,
  early_exaggeration_iter = 0L,
  learning_rate = 0.1,
  early_exaggeration = 4,
  exaggeration = 1.5,
  initial_momentum = 0.8,
  final_momentum = 0.8,
  max_grad_norm = 0.25,
  max_step_norm = Inf,
  n_negatives = NULL,
  exact_repulsion_threshold = 4096L,
  n_threads = NULL,
  seed = 4L,
  backend = "auto",
  verbose = FALSE
)
```

Arguments

reference_layout	Numeric reference embedding matrix, or a ‘fastEmbedR_embedding’ object.
knn	Optional query-to-reference KNN list with ‘indices’ and ‘distances’.
reference_data	Reference observations in the same preprocessing space used to fit ‘reference_layout’. Required only when ‘knn’ is ‘NULL’.

<code>new_data</code>	Query observations in the same preprocessing space as <code>'reference_data'</code> . Required only when <code>'knn'</code> is <code>'NULL'</code> .
<code>k</code>	Number of reference neighbors used for transform KNN. If <code>'NULL'</code> , uses at least <code>'3 * perplexity'</code> , matching the usual t-SNE affinity width.
<code>perplexity</code>	Transform perplexity. The openTSNE default is 5.
<code>initialization</code>	Initial query placement: <code>"median"</code> from reference neighbors, <code>inverse-distance</code> <code>"weighted"</code> , or <code>"random"</code> .
<code>Y_init</code>	Optional initial query embedding matrix. When supplied, it overrides <code>'initialization'</code> .
<code>n_iter</code>	Number of normal transform optimization iterations.
<code>early_exaggeration_iter</code>	Number of early-exaggeration iterations.
<code>learning_rate</code>	Transform learning rate.
<code>early_exaggeration</code>	Early exaggeration multiplier.
<code>exaggeration</code>	Normal transform exaggeration multiplier.
<code>initial_momentum</code>	Momentum used during early exaggeration.
<code>final_momentum</code>	Momentum used during normal optimization.
<code>max_grad_norm</code>	Maximum per-query gradient norm. Use <code>'Inf'</code> to disable.
<code>max_step_norm</code>	Maximum per-query step norm. Use <code>'Inf'</code> to disable.
<code>n_negatives</code>	Number of sampled reference points for repulsion when the reference set is larger than <code>'exact_repulsion_threshold'</code> . GPU sampled repulsion is native and experimental for large reference sets.
<code>exact_repulsion_threshold</code>	Use exact query-reference repulsion at or below this reference count.
<code>n_threads</code>	Number of CPU worker threads for the native optimizer.
<code>seed</code>	Random seed.
<code>backend</code>	Backend used for query KNN when <code>'knn'</code> is <code>'NULL'</code> ; <code>"metal"</code> and <code>"cuda"</code> run their native fixed-reference transform optimizers when those backends were compiled. Explicit unavailable GPU requests fail; CPU work is never reported as Metal or CUDA.
<code>verbose</code>	Print native optimizer progress.

Value

A numeric matrix with one row per query observation.

Examples

```
reference <- matrix(c(0, 0, 1, 0, 0, 1, 1, 1), 4L, 2L, byrow = TRUE)
query_knn <- list(
  indices = matrix(c(1L, 2L, 3L, 4L), 2L, 2L, byrow = TRUE),
  distances = matrix(c(0.1, 0.2, 0.15, 0.25), 2L, 2L, byrow = TRUE)
)
query_layout <- transform_tsne(
  reference, knn = query_knn, perplexity = 0.5,
  n_iter = 2, exact_repulsion_threshold = 10, seed = 1
)
```

umap

*Run UMAP from data or precomputed KNN***Description**

`umap()` computes or reuses a nearest-neighbor graph and then calls `umap_knn`. Quality metrics are computed explicitly with `evaluate_embedding` rather than inside the embedding call.

Usage

```
umap(
  data,
  n_neighbors = NULL,
  n_components = 2L,
  standardize = FALSE,
  pca_dims = NULL,
  metric = c("euclidean", "cosine", "correlation", "inner_product"),
  nn = NULL,
  seed = 4L,
  backend = c("cpu", "cuda", "metal"),
  n_threads = NULL,
  keep_knn = FALSE,
  graph_mode = c("binary", "fuzzy"),
  verbose = FALSE
)
```

Arguments

<code>data</code>	Numeric matrix/data frame, or a KNN object with indices and distances fields.
<code>n_neighbors</code>	Number of non-self neighbors. NULL chooses the package default for the data size.
<code>n_components</code>	Output dimensionality.
<code>standardize</code>	Center and scale columns before KNN when data is a matrix. Defaults to FALSE so one-call results match a KNN object computed from the supplied matrix.
<code>pca_dims</code>	Optional PCA dimension before KNN.
<code>metric</code>	KNN distance metric for one-call matrix input: "euclidean", "cosine", "correlation", or "inner_product".
<code>nn</code>	Optional precomputed KNN result when data is a matrix.
<code>seed</code>	Random seed.
<code>backend</code>	Execution backend: "cpu", "cuda", or "metal". CPU KNN uses package-native HNSW. Metal uses package-native exact or recall-tuned IVF-Flat search. CUDA uses direct FAISS GPU exact search below 100,000 rows and direct RAPIDS cuVS IVF-Flat above that threshold; the KNN result stays on the device through graph construction and optimization. GPU requests must resolve to a real native backend; the package does not relabel CPU work as GPU.
<code>n_threads</code>	Number of CPU worker threads for KNN and CPU UMAP.
<code>keep_knn</code>	If TRUE, retain KNN matrices in the returned object.

graph_mode	Graph weighting mode. "binary" uses a symmetric unit-weight graph as an adjacency-only approximation; it is not standard UMAP or necessarily faster. "fuzzy" uses standard UMAP fuzzy graph weights and should be used for reference-implementation comparisons.
verbose	Print progress.

Value

A fastEmbedR_embedding object.

Examples

```
x <- scale(as.matrix(iris[, 1:4]))
fit <- umap(x, n_neighbors = 15, seed = 1)
plot(fit)
```

umap_init	<i>Precompute a Reusable UMAP Initialization</i>
-----------	--

Description

Separates UMAP neighbor search, graph construction, and initialization from stochastic optimization. This permits repeated CPU, Metal, or CUDA optimizer runs from the same graph and coordinates without recomputing either stage.

Usage

```
umap_init(
  x,
  distances = NULL,
  n_neighbors = NULL,
  n_components = 2L,
  metric = c("euclidean", "cosine", "correlation", "inner_product"),
  backend = c("cpu", "cuda", "metal"),
  seed = 4L,
  n_threads = NULL,
  graph_mode = c("binary", "fuzzy")
)
```

Arguments

x	Numeric data, a KNN object, or an object returned by prepare_umap_knn().
distances	Optional KNN distance matrix when x is an index matrix.
n_neighbors	Number of non-self neighbors when KNN must be computed.
n_components	Output dimensionality.
metric	Distance metric used only when KNN must be computed.
backend	Initialization backend: "cpu", "cuda", or "metal".
seed	Random seed.
n_threads	Number of CPU worker threads.
graph_mode	UMAP graph weighting: "binary" or "fuzzy".

Details

For a fuzzy graph, native Metal or CUDA spectral kernels are used when the corresponding backend is requested and available. Binary initialization uses the same unit-weight CSR graph consumed by the optimizer. Explicit unavailable GPU backends fail rather than silently falling back to CPU.

CUDA KNN input is copied to the host only for this standalone diagnostic API. The ordinary one-call CUDA UMAP workflow retains its device-resident path.

Value

A `fastEmbedR_umap_initialization` object containing the initial layout, prepared KNN/CSR graph, resolved parameters, and component timings. The object can be passed directly to `umap_knn()`.

Examples

```
x <- scale(as.matrix(iris[, 1:4]))
initialization <- umap_init(
  x,
  n_neighbors = 15,
  backend = "cpu",
  graph_mode = "fuzzy",
  seed = 1
)
layout <- umap_knn(initialization, backend = "cpu", seed = 1)
plot(layout, pch = 21, bg = iris$Species)
```

umap_knn

Run UMAP from precomputed KNN

Description

`umap_knn()` embeds observations from a supplied nearest-neighbor graph using the stable native UMAP path. It is the KNN-input UMAP entry point used by `embed_knn` with `method = "umap"`.

Usage

```
umap_knn(
  indices,
  distances = NULL,
  n_components = 2L,
  seed = 42L,
  verbose = FALSE,
  backend = c("cpu", "cuda", "metal"),
  n_threads = NULL,
  graph_mode = c("binary", "fuzzy")
)
```

Arguments

<code>indices</code>	A KNN object, or an integer KNN index matrix. A self-neighbor first column is removed automatically.
<code>distances</code>	Numeric KNN distance matrix matching indices. Leave NULL when indices is an <code>nn()</code> result.

n_components	Output dimensionality.
seed	Random seed.
verbose	Print native optimizer progress.
backend	Execution backend: "cpu", "cuda", or "metal". GPU requests must resolve to a real native backend; the package does not label CPU work as GPU.
n_threads	Number of CPU worker threads for CPU UMAP.
graph_mode	Graph weighting mode. "binary" uses a symmetric unit-weight graph. "fuzzy" uses standard UMAP fuzzy graph weights.

Value

A numeric embedding matrix with resolved settings in `attr(layout, "fastEmbedR_config")`.

Examples

```
x <- scale(as.matrix(iris[, 1:4]))
d <- as.matrix(stats::dist(x))
diag(d) <- Inf
k <- 10L
idx <- t(apply(d, 1L, order))[, seq_len(k), drop = FALSE]
dst <- matrix(d[cbind(rep(seq_len(nrow(d)), each = k), as.vector(t(idx)))],
  nrow = nrow(d), byrow = TRUE)
layout <- umap_knn(idx, dst, seed = 1)
plot(layout, pch = 21, bg = iris$Species)
```